

ASSIGNMENT 2: ASYMPTOTIC NOTATION & DIVIDE AND CONQUER

Instructor: Orhan Özgüner

Due: October 1 before 11:59 PM

Problem 1

For each of the following pairs of functions, write down the asymptotic relation between $f(n)$ and $g(n)$; *i.e.*, if $f(n) = x(g(n))$ where $x \in \{o, \Theta, \omega, O, \Omega\}$. Assume that $k \geq 0, \epsilon > 0$, and $c > 1$. Provide a justification for your answer.

	$f(n)$	$g(n)$
a)	$\log^k n$	n^ϵ
b)	n^k	c^n
c)	$\sqrt{n}$	$n^{\sin(n)}$

Problem 1 Solutions

Part A

In order to compare these two functions, I set up the following relation $f(n) < g(n)$

$(\log_2(n))^k$	vs	n^ϵ	Stated the relation.
$\log_2(n)$	vs	$n^{\frac{\epsilon}{k}}$	Exponentiated each side by $\frac{1}{k}$
$n^{\frac{1}{n}}$	vs	$2^{\frac{n\epsilon}{kn}}$	Exponentiated each side by $\frac{2}{n}$
$n^{\frac{1}{n}}$	vs	$2^{\frac{\epsilon}{k}}$	Simplified expression
$n^{\frac{k}{n}}$	vs	2^ϵ	Simplified expression

The left term's power approaches 1 as n approaches infinity. Epsilon is greater than 0, so the right side of the relationship will always be greater than 1. Therefore, $n^{\frac{k}{n}} < 2^\epsilon$, giving us the asymptotic relationship:

$$\log_2^k(n) = o(n^\epsilon)$$

Part B

In this case, A is a polynomial function and B is an exponential function. From the Cormen textbook, we know that the following limit is true:

$$\lim_{n \rightarrow \infty} \frac{n^k}{c^n} = 0$$

Since this limit evaluates to 0, the time complexity relationship is:

$$n^k = o(c^n)$$

Part C

In order to compare this pair, we can express these functions as:

$$\begin{array}{cc} \sqrt{n} & n^{\sin n} \\ n^{\frac{1}{2}} & n^{\sin n} \\ \frac{1}{2}(\log_2(n)) & \sin n \log_2(n) \end{array}$$

Neither one of these multiplying factors is definitively greater than the other. The sine function at times is greater than $\frac{1}{2}$, but since it is an oscillating function there is never any point where it will *always* be. Since this isn't asymptotic, none of these options can accurately determine this pair. $\sin(n)$ will continuously oscillate higher and lower than $\frac{1}{2}$.

No relationship based on given options

Problem 2

Considering functions $f(n) \geq 0$, $g(n) \geq 0$, and constant $c > 0$, indicate whether each of the following statements is true. Prove the statements that are true by providing a formal argument that is based on the definition of asymptotic notation. For statements that are false, provide a counter-example to prove that they are false.

(a) If $f(n) \geq 1$, then $f(n) + c = O(f(n))$.

(b) $f(2n) = \Theta(f(n))$.

(c) If $f(n) = O(n^c)$, then $f(2n) = O(n^c)$.

Problem 2 Solutions

(a) If $f(n) \geq 1$, then $f(n) + c = O(f(n))$.

Proof. We need to show that $f(n) + c \leq f(n)d$ for some constant $d > 0$. That is,

$$\begin{aligned} f(n) + c &\leq f(n)d \\ c &\leq f(n)(d - 1) \\ \frac{c}{d - 1} &\leq f(n) \end{aligned} \tag{1}$$

Hence, Equation (1) is satisfied if $\frac{c}{d - 1} = 1$ since $f(n) \geq 1$. Thus, it is satisfied for $d = c + 1$. $\square$

(b) $f(2n) = \Theta(f(n))$.

Disproof. Let $f(n) = 2^n$. Then, $f(2n) = 2^{(2n)} = 4^n$. If the statement is correct, $f(2n) = \Theta(f(n))$ which is not satisfied since $2^n \neq \Theta(4^n)$. $\square$

(c) If $f(n) = O(n^c)$, then $f(2n) = O(n^c)$.

Proof. We need to show that $f(2n) \leq kn^c$ for some $k > 0$.

If $f(n) = O(n^c)$, then there is a constant $d > 0$ such that $f(n) \leq dn^c$. Thus,

$$\begin{aligned} f(n) &\leq n^c d \\ f(2n) &\leq (2n)^c d \\ f(2n) &\leq n^c 2^c d \end{aligned} \tag{2}$$

Hence, it is satisfied for $k = 2^c d$. $\square$

Problem 3

Let $0 < \lambda < 1 < a < b$ be constants. Solve the following recurrences using Master Method, noting the case that applies.

(a) $T(n) = b T(\frac{n}{a}) + \Theta(n)$.

(b) $T(n) = a^2 T(\frac{n}{a}) + \Theta(n^2)$.

(c) $T(n) = T(\lambda n) + n^\lambda$.

Problem 3 Solutions

(a) $T(n) = b T(\frac{n}{a}) + \Theta(n)$.

In order to see if the Master Method is applicable, we need to compare $n^{\log_a(b)}$ and $f(n) = \Theta(n)$. Since $b > a$, $\log_a(b) > 1$. Thus, $f(n) = \Theta(n)$ is polynomially smaller than $n^{\log_a(b)}$ and the case 1 of Master Method applies. Hence, $T(n) = \Theta(n^{\log_a(b)})$.

(b) $T(n) = a^2 T(\frac{n}{a}) + \Theta(n^2)$.

In order to see if the Master Method is applicable, we need to compare $n^{\log_a(a^2)}$ and $f(n) = \Theta(n^2)$. Since $n^{\log_a(a^2)} = n^2$, $f(n) = \Theta(n^2) = \Theta(n^{\log_a(a^2)})$. Thus, the case 2 of Master Method applies. Hence, $T(n) = \Theta(n^{\log_a(a^2)} \log n) = \Theta(n^2 \log n)$.

(c) $T(n) = T(\lambda n) + n^\lambda$.

In order to see if the Master Method is applicable, we need to compare $n^{\log_{\frac{1}{\lambda}}(1)}$ and $f(n) = n^\lambda$. Since $\log_{\frac{1}{\lambda}}(1) = 0$, $n^{\log_{\frac{1}{\lambda}}(1)} = 1$. Moreover, $f(n) = n^\lambda$ is polynomially larger than $n^{\log_{\frac{1}{\lambda}}(1)} = 1$ since $\lambda > 0$. Thus, the case 3 of Master Method might be applicable (if the regularity condition is satisfied). For the regularity condition we need to show,

$$f(\lambda n) \leq c f(n) \quad (3)$$

for some $c < 1$ for all sufficiently large n . Since $f(n) = n^\lambda$, Equation 3 becomes:

$$\begin{aligned} f(\lambda n) &\leq c f(n) \\ (\lambda n)^\lambda &\leq c n^\lambda \\ \lambda^\lambda n^\lambda &\leq c n^\lambda \\ \lambda^\lambda &\leq c \end{aligned} \quad (4)$$

Also, $\lambda^\lambda < 1$ because $0 < \lambda < 1$. Thus, Equation 3 is satisfied for $c = \lambda^\lambda < 1$.

Hence, the case 3 of Master Method is applicable and $T(n) = \Theta(f(n)) = \Theta(n^\lambda)$.

Problem 4

You are given an array of k sorted arrays each of which has a length $\frac{n}{k}$ elements. Describe an efficient algorithm to merge these arrays to obtain one sorted array of length n .

Problem 4 Solutions

Explanation: We can do this using a divide-and-conquer approach by recursively merging $\frac{k}{2}$ pairs of arrays of length $\frac{n}{k}$ to create $\frac{k}{2}$ sorted arrays of length $\frac{2n}{k}$ each, then merging in pairs to create $\frac{k}{4}$ arrays of length $\frac{4n}{k}$ etc. We use the MERGE procedure from the MERGE-SORT algorithm to merge each pair of arrays.

Pseudocode

Assume 0-based indexing is used for this problem.

Algorithm 1 Merge K-sorted Arrays

Require: arr is array of k sorted arrays, each of length $\frac{n}{k}$

```
procedure MERGEKARRAYS( $arr$ )  
    return MERGE-RECURSION( $arr$ , 0, length( $arr$ ) - 1)  
end procedure  
  
procedure MERGE-RECURSION( $arrlist, i, j$ )  
    if  $i = j$  then  
        return  $arrlist[i]$   
    else  
         $mid \leftarrow \lfloor \frac{i+j}{2} \rfloor$   
         $left \leftarrow$  MERGE-RECURSION( $arrlist, i, mid$ )  
         $right \leftarrow$  MERGE-RECURSION( $arrlist, mid + 1, j$ )  
        return MERGEARRAYS( $left, right$ )  
    end if  
end procedure  
  
procedure MERGEARRAYS( $left, right$ )  
     $i, j, k \leftarrow 0$   
     $n_1, n_2 \leftarrow$  length( $left$ ), length( $right$ )  
     $output \leftarrow$  array of length  $n_1 + n_2$   
     $left[n_1] \leftarrow \infty$   
     $right[n_2] \leftarrow \infty$   
    while  $k < n_1 + n_2$  do  
        if  $left[i] < right[j]$  then  
             $output[k] \leftarrow left[i]$   
             $i \leftarrow i + 1$   
        else  
             $output[k] \leftarrow right[j]$   
             $j \leftarrow j + 1$   
        end if  
         $k \leftarrow k + 1$   
    end while  
    return  $output$   
end procedure
```

Proof of Correctness: Merge-Recursion

Inductive Statement: The array returned is sorted and holds all elements from arrays $arrlist[i..j]$.

Base Case: When $i = j$, this means the length of the subarray being considered is 1. Based on the assumptions of the problem, $arrlist[i]$ is already sorted. Therefore, the inductive statement holds true.

Inductive Step: Assume the inductive statement holds for some $j - i \geq 0$. This means the calls to MERGE-RECURSION guarantee that *left* and *right* are both sorted. We prove below that the call to merge results in a sorted list containing all elements from *left* and *right*. Since *left* and *right* contain all elements from arrays in *arrlist*[*i..j*], the inductive statement holds true.

Proof of Correctness: merge

Loop Invariant

1. *output*[0..*k* - 1] is sorted and contains the *k* smallest elements from *left* and *right* combined.
2. *output*[*k* - 1] $\leq$ *left*[*i..n*₁], *right*[*j..n*₂]

- **Initialization** Before the loop, $k = 0$ which means that element *output*[-1] does not exist. This means statement (1) and (2) of the L.I. are vacuously true.
- **Maintenance** Assume the L.I. holds for some arbitrary iteration of the loop. There are two cases at hand:

Case 1: *left*[*i*] $\leq$ *right*[*j*]

In the body, *output*[*k*] = *left*[*i*], *i* is incremented, and *k* is incremented. Previously, *output*[*k* - 1] $\leq$ *left*[*i*], so now statement (1) of the loop invariant will hold. Statement (2) of the L.I. will hold since *left* and *right* are sorted, and *output*[*k* - 1] $<$ *right*[*j*] from the branch statement.

Case 2: *B*[*i*] $>$ *C*[*j*]

Statements (1) and (2) of the L.I. hold by a similar argument made in Case 1.

- **Termination** When the loop terminates $k = n_1 + n_2$. By the L.I., *output*[0..*n*₁ + *n*₂ - 1] is sorted and contains the *n*₁ + *n*₂ smallest elements from both *left* and *right* (which is all of the elements). This means we have achieved the goal of merging the sorted subarrays together.

Runtime Analysis

Students cannot use the Master Method here since there are two variables, *k* and *n*. Therefore, the recurrence equation doesn't follow the definition of a recurrence solvable by the Master Method. We divide the arrays and recursively call to sort. After $\log k$ steps, we will obtain one sorted array length $\frac{n}{k}$. The i^{th} level of the algorithm performs 2^i merge operations on 2^{i+1} arrays. Each array has $\frac{n}{2^{i+1}}$ elements, therefore the work per level is n . Thus, the total runtime is $\Theta(n \log k)$.